

CV Optimizer

Architecture & Project Audit

7 Agent AI Pipeline - LangGraph-style Orchestration - Production Status

GPT-4o Multi-Agent

AbortSignal Cancellation

MongoDB Atlas RAG

ENABLE_RAG Toggle

TypeScript · Clean Build

Project Audit — Current Status

Full audit run against every source file in both frontend and backend. TypeScript compiles clean on both sides with zero errors.

Check	Result	Notes
Backend tsc --noEmit	✓ PASS	0 errors · all 7 agents + orchestrator + routes
Frontend tsc --noEmit	✓ PASS	0 errors · React + Vite + shadcn/ui

Feature Completeness

7-Agent pipeline (Aria→Marcus→Nora→Leo→Sage→Rex→Vera)	COMPLETE	All agents have index.ts + schema.ts + prompt.ts + 4 definition .md files
File-based agent definitions (IDENTITY / SOUL / GOALS / CONSTRAINTS)	COMPLETE	PromptLoader.ts compiles 4 static files into system prompt at startup; cached
MEMORY.md per-agent lesson system	COMPLETE	MemoryManager.ts reads/writes MEMORY.md; orchestrator calls recordLesson() before re-runs
Zod schema validation (Rex) after every agent step	COMPLETE	ValidatorAgent runs synchronously inside withRetry wrapper; no OpenAI call
Quality judge (Vera) with selective agent re-run	COMPLETE	Vera scores 0-10; if < JUDGE_QUALITY_THRESHOLD, orchestrator re-runs only flagged agents
Exponential backoff retry (withRetry)	COMPLETE	400ms→800ms→1600ms; checks abort signal before each sleep; MAX_RETRIES from env
Client disconnect → pipeline cancellation	COMPLETE	Dedicated Cancellation.ts module; AbortSignal stored in PipelineState; attachSignal() on each agent; req.once('aborted') + res.once('close') dual guard; 499 response
MongoDB Atlas RAG layer	COMPLETE	KnowledgeRetriever + Embedder + VectorStore; 3 parallel prefetch queries; graceful empty-result fallback
ENABLE_RAG env toggle	COMPLETE	env.ts Zod schema; cross-field check (MONGODB_URI required only when ENABLE_RAG=true); guards in mongodb.ts + all 3 KnowledgeRetriever functions + /health endpoint
Backend API endpoint POST /api/analyze	COMPLETE	Multer 5MB limit; PDF/DOCX/TXT filter with clean 400 errors; full pipeline wiring
Frontend upload zone + loading spinner	COMPLETE	Always-visible upload form; Loader2 spinner during analysis; results scroll into view below
Frontend mock data	REMOVED	api.ts, vite-env.d.ts, .env all cleaned. mockData.ts neutralised to empty stub
Backend → Frontend response mapping	COMPLETE	mapResponse() in api.ts bridges nested backend shape → flat AnalysisResult UI expects

Dead code: LoadingScreen.tsx	MINOR	Component exists but is not imported anywhere — leftover from old 3-phase state machine. Safe to delete; no compile impact
Dead code: mockData.ts	MINOR	Empty stub file (export {} only). Not imported. Safe to delete
RAG collections seeded	PENDING	Collections start empty. Agents fall back to GPT-4o knowledge. No blocker — seed via Atlas UI when ready
Unit / integration tests	NONE	No test files exist. For production: add Jest tests for Zod schemas, mapResponse(), and orchestrator retry logic

Table 1 — Full feature audit. MINOR items have zero runtime impact.

1. Why Multi-Agent Instead of One Big Prompt?

The simplest possible implementation is a single GPT-4o call: send the CV and job title, receive back a JSON blob with scores and recommendations. One request, done. So why didn't we build it that way?

Context overload: A prompt that must extract structured data, analyse role requirements, detect gaps, compute weighted scores, and generate prioritised recommendations simultaneously is asking one model call to hold too many competing objectives. In practice it satisfies some goals and quietly degrades others — usually the scoring formula and the specificity of recommendations.

No intermediate checkpoints: If Aria misreads the skills section and that error isn't caught before Nora runs the gap analysis, every downstream output is corrupted. With a single prompt there is no checkpoint — one bad extraction cascades into a wrong score and wrong recommendations with no way to detect it.

Untargeted recovery: If the output quality is low, you must retry the entire prompt, burning 5x the tokens and taking the same time as the original run. With separate agents you re-run only the one that failed — Vera tells the orchestrator exactly which agent needs correction.

Separation of concerns: Each task has a distinct identity, a specific output schema, and different failure modes. Mixing them into one prompt makes it impossible to tune, debug, or improve any one part in isolation. Changing Leo's scoring formula in a multi-agent system is one file; in a single prompt it risks breaking the extraction and recommendation sections.

No within-run learning: When Vera finds Leo scored too generously, there is nowhere in a single prompt to record that lesson and apply it before re-running the affected step. With agents, the orchestrator writes to MEMORY.md and the corrected context is loaded on the retry.

	Single GPT-4o Call	7-Agent Pipeline
Mid-run validation	None	Rex (Zod) after every step
Recovery on failure	Retry entire prompt	Re-run only the failing agent
Token cost on retry	Full prompt × retries	1 agent prompt × retries
Traceability	Black-box single output	Per-agent logs + state object
Within-run memory	No	MEMORY.md per agent
Tuning a single step	Edit the entire monolith	Edit one agent's .md files
Abort on disconnect	Request hangs until done	AbortSignal cancels at OpenAI layer

Table 2 — Single prompt vs. 7-agent pipeline. Green column = chosen approach.

2. Direct Agent-to-Agent vs. Centralized Orchestration

Once multiple agents are the answer, there are two ways to wire them together. This is the most important architectural decision in the codebase.

Option A — Direct (Decentralized): Aria calls Marcus calls Nora...

Each agent imports and calls the next one directly. The pipeline is implicit in the call chain.

- Agents import each other → circular dependency risk as the number grows.
- Validation (Rex) must be wired inside every agent that calls the next — not orthogonal.

- AbortSignal must be threaded manually through every agent's call to the next.
- No global state: a late-stage agent cannot access cvText or ragContext without it being forwarded through every hop.
- Reordering the pipeline means editing agent code, not a wiring file.
- Testing Nora in isolation requires stubbing Marcus's output manually.

Option B — Centralized (LangGraph-style): What we built

PipelineOrchestrator owns the run. It drives the loop, manages state, injects validation, attaches the abort signal, and triggers re-runs. Agents are pure, isolated nodes.

- ✓ **Agents are pure functions:** `run(state) → state`. No knowledge of predecessor or successor. Fully testable in isolation.
- ✓ **Single shared PipelineState:** cvText, ragContext, every agent's output, abort signal, error log — all in one object that any agent can read from. No passing context through parameter chains.
- ✓ **Validation is orthogonal:** Rex plugs in once at the orchestrator level. No agent knows Rex exists. Adding a new agent automatically gets validation without any code change in that agent.
- ✓ **AbortSignal via setExecutionSignal():** The orchestrator calls `attachSignal(node, state.abortSignal)` before every `node.run()`. BaseAgent stores it as `this.executionSignal` and passes it to every `this.chat()` call. The signal reaches the OpenAI SDK's HTTP layer for immediate in-flight cancellation.
- ✓ **Declarative wiring:** `orchestrator.use(new Aria()).use(new Marcus())...` Adding, removing, or reordering agents is a single line change in `routes/analyze.ts`.
- ✓ **Selective quality re-runs:** Vera returns agent names to re-run. The orchestrator loops only over those. Agents don't know they're being re-run — the orchestrator handles the lesson injection and re-judgment automatically.

3. The 7 Agents

Each agent is defined by four Markdown files (IDENTITY · SOUL · GOALS · CONSTRAINTS) that PromptLoader compiles into its system prompt at startup and caches. Changing an agent's behaviour means editing a text file, not TypeScript.

Aria CVProfilerAgent

Reads raw CV text → structured CVProfile. Extracts: name, contact, skills list, experience entries with role/company/highlights, education, certifications, projects, languages, links, total years experience, and a list of every section heading present.

Marcus RoleAnalyzerAgent

Job title → RoleProfile. Uses GPT-4o knowledge + optional Atlas RAG context. Outputs: required/nice-to-have skills, years required, expected CV sections, ATS keywords, seniority level (junior/mid/senior/principal), industry context.

Nora GapDetectorAgent

Aria's CVProfile × Marcus's RoleProfile → GapData. Classifies every required skill as: existing (evidenced in job context), missing (zero presence), or partial (listed but no practical context — most common mistake in CVs). Also flags missing sections and experience gaps.

Leo ScoreEngineAgent

Full state → ScoreData. Applies a deterministic weighted formula: $\text{matchScore} = (\text{technicalSkills} \times 0.40) + (\text{experience} \times 0.30) + (\text{presentation} \times 0.15) + (\text{keywords} \times 0.15)$. Each category has a one-line note. Verdict is exactly 2-3 sentences.

Sage RecommenderAgent

Gaps + scores → RecommendationsData. Every recommendation has priority (High/Medium/Low), a short scannable action, a specific detail referencing the actual gap, and the expected impact. Minimum 6 recommendations, minimum 2 per priority level.

Rex ValidatorAgent

Does NOT call OpenAI. Pure Zod schema validation. Runs after every agent step inside the withRetry wrapper. If the output doesn't conform to the TypeScript interface, the orchestrator flags a retry. Zero cost, zero latency.

Vera QualityJudgeAgent

Runs once after all 5 main agents complete. Reviews the entire output for logical consistency: are existingSkills actually in the CV? Does matchScore match the formula (± 2 pts)? Are there ≥ 2 recommendations per priority? Scores 0-10. If below JUDGE_QUALITY_THRESHOLD (default 7), returns agent names + issues for re-run.

4. Orchestration Flow — Step by Step

From the moment the user clicks Analyze CV to results rendering below the upload form.

1. HTTP POST + Abort Setup

Frontend POSTs multipart/form-data (file + jobTitle) to /api/analyze. Route immediately creates an AbortController and attaches two guards: req.once('aborted') and res.once('close'). Either one fires abort.abort(). The signal is stored in PipelineState.abortSignal for the orchestrator.

2. File Parsing

Multer buffers the upload in memory (5MB limit, PDF/DOCX/TXT only — clean 400 on violations). extractText() dispatches to pdf-parse for PDFs, mammoth for DOCX, or direct read for TXT. Result: a plain string of CV text.

3. RAG Prefetch (optional)

If ENABLE_RAG=true: three Atlas Vector Search queries fire in parallel via Promise.all — role context for Marcus, skill context for Nora, improvement patterns for Sage. If ENABLE_RAG=false or collections are empty: all three return "" instantly. Agents fall back to GPT-4o training knowledge with no code change required.

4. State Initialisation

createInitialState() produces PipelineState: runId, cvText, jobTitle, startedAt, abortSignal, ragContext map (pre-seeded with the 3 prefetch results), and empty slots for each agent's output. This is the single source of truth.

5. Agent Loop (Aria → Marcus → Nora → Leo → Sage)

For each of the 5 main agents: (a) throwIfAborted() checks the signal, (b) attachSignal() calls node.setExecutionSignal(signal) so BaseAgent.chat() passes it to OpenAI, (c) node.run(state) is called inside withRetry(), (d) Rex validates the output shape with Zod immediately after, (e) if validation fails: withRetry retries up to MAX_RETRIES times with exponential backoff (400→800→1600ms), checking the abort signal before each sleep.

6. Quality Judge (Vera)

Vera reviews the complete assembled output and returns a JudgeVerdict with a score 0-10. If score < JUDGE_QUALITY_THRESHOLD: the orchestrator reads verdict.issues, writes targeted lessons to each failing agent's MEMORY.md via recordLesson(), then re-runs only the listed agents inside withRetry(). Vera runs again after corrections.

7. Response

AnalysisResult assembled from all agent outputs and returned as JSON. The frontend's mapResponse() in api.ts flattens the nested backend shape (scoreData.categoryScores, gapData.experienceGaps, recommendations.recommendations) into the flat AnalysisResult that all UI components expect. Results scroll smoothly into view below the upload zone — no page transition.

5. Client Disconnect → Pipeline Cancellation

Without this, a browser refresh leaves an orphaned pipeline running on the backend — burning OpenAI tokens, holding state in memory — with nobody waiting for the result.

Step	Component	What Happens
Browser refresh / close	Browser	TCP connection closes. Node fires 'aborted' or 'close' event on the request.
Dual event guard	<code>routes/analyze.ts</code>	<code>req.once('aborted') + res.once('close')</code> both call <code>abortRun()</code> . A guard prevents double-abort.
Signal propagated	<code>PipelineState</code>	<code>abortSignal</code> stored on state. <code>throwIfAborted()</code> is called at every phase boundary.
Agent signal injection	<code>PipelineOrchestrator</code>	<code>attachSignal()</code> calls <code>node.setExecutionSignal(signal)</code> before every <code>node.run()</code> . <code>BaseAgent</code> stores it as <code>this.executionSignal</code> .
OpenAI call cancelled	<code>BaseAgent.chat()</code>	<code>this.chat()</code> passes <code>this.executionSignal</code> to the OpenAI SDK. The SDK cancels the HTTP request at network level — no more tokens consumed.
Retry sleep aborted	<code>withRetry()</code>	<code>sleep()</code> listens for the abort event. Promise rejects immediately with <code>PipelineAbortedError</code> instead of waiting the full backoff duration.
Clean 499 response	<code>routes/analyze.ts</code>	<code>isAbortError(err)</code> catch returns HTTP 499 (client closed request). No 500 logged. No error alert to the user.

6. RAG Toggle — `ENABLE_RAG`

A single env flag makes MongoDB and the entire RAG layer optional. The server runs cleanly either way — no code changes required.

<code>ENABLE_RAG</code>	<code>MONGODB_URI</code>	Behaviour
<code>true</code> (default)	Set	Full RAG — Atlas Vector Search prefetch active
<code>true</code>	Missing	Server refuses to start (env validation fails with clear error)
<code>false</code>	Set	MongoDB ignored. RAG silently skipped. GPT-4o knowledge only.
<code>false</code>	Missing	Clean start. No Atlas connection. Zero config required.

File	Change
<code>config/env.ts</code>	<code>ENABLE_RAG: z.coerce.boolean().default(true) + cross-field check</code>
<code>config/mongodb.ts</code>	<code>connectMongoDB()</code> is a no-op when <code>ENABLE_RAG=false</code>
<code>rag/KnowledgeRetriever.ts</code>	All 3 functions (<code>getRoleContext/getSkillContext/getImprovementContext</code>) return "" immediately when <code>ENABLE_RAG=false</code>
<code>routes/analyze.ts</code>	Log line on startup when RAG is disabled
<code>server.ts</code>	<code>/health</code> response includes <code>rag: 'enabled' 'disabled'</code>

7. Technology Stack

Layer	Technology	Reason
Frontend	React 18 + Vite + TS	Fast HMR, strict typing, component model
Styling	Tailwind v4 + shadcn/ui	Utility-first; pre-built accessible components
API client	Fetch + FormData	Native browser APIs; no extra dependency
Backend	Node.js + Express + TS	Minimal overhead; great ecosystem; strict types
AI model	OpenAI GPT-4o	Best JSON-mode output quality; AbortSignal support
Embeddings	text-embedding-3-small	Fast, cheap, 1536-dim; good semantic similarity
Vector DB	MongoDB Atlas	Vector search + document store in one service
Validation	Zod	Runtime schema validation + TypeScript inference
File parsing	pdf-parse + mammoth	PDF and DOCX text extraction; no native binaries
Logging	Winston	Structured logs; multiple transports
Rate limiting	express-rate-limit	Protects OpenAI quota from burst abuse
Cancellation	AbortController / Signal	Web-standard API; OpenAI SDK natively supports it
Env validation	Zod (cross-field checks)	Fails fast on startup; clear error messages

8. Key File Structure

backend/ src/ agents/ base/ BaseAgent.ts PromptLoader.ts MemoryManager.ts CVProfilerAgent/ RoleAnalyzerAgent/ GapDetectorAgent/ ScoreEngineAgent/ RecommenderAgent/ ValidatorAgent/ QualityJudgeAgent/ orchestrator/ PipelineOrchestrator.ts AgentNode.ts Cancellation.ts RetryPolicy.ts rag/ KnowledgeRetriever.ts Embedder.ts VectorStore.ts config/	 ← abstract base; setExecutionSignal(); this.chat() ← compiles 4 .md files into system prompt (cached) ← reads / appends MEMORY.md per agent ← index.ts · schema.ts · IDENTITY/SOUL/GOALS/CONSTRAINTS.md ← same structure ← same structure ← same structure ← same structure ← Zod-only; no OpenAI call ← reviews full output; returns rerunAgents[] ← drives loop; attachSignal(); judge re-run logic ← interface { name; run(state) } ← PipelineAbortedError · isAbortError · throwIfAborted ← withRetry(); abortable sleep() ← 3 retrieval fns; ENABLE_RAG guards ← text-embedding-3-small wrapper ← Atlas Vector Search query helper
---	---

■ ■ ■■ env.ts	← Zod schema; ENABLE_RAG cross-field check
■ ■ ■■ mongodb.ts	← connectMongoDB(); no-op when ENABLE_RAG=false
■ ■ ■■ openai.ts	← chat(system, user, signal?); embed()
■ ■■ routes/	
■ ■ ■■ analyze.ts	← POST /api/analyze; AbortController; 499 handling
■ ■■ types/	
■ ■■ agents.ts	← CVProfile · RoleProfile · GapData · ScoreData · etc.
■ ■■ analysis.ts	← AnalysisResult (final API response shape)
■ ■■ pipeline.ts	← PipelineState (carries abortSignal)
frontend/	
■ ■■ src/	
■■■ components/	
■ ■■ UploadZone.tsx	← file drop + job title + Loader2 spinner
■ ■■ ResultsDashboard.tsx	← top-level results container
■ ■■ ui/	← shadcn/ui components
■■■ services/	
■ ■■ api.ts	← analyzeCV(); mapResponse() flattens backend shape
■■■ types.ts	← AnalysisResult (flat frontend shape)

Every design decision in this codebase has a concrete reason: separate agents for separation of concerns, a centralized orchestrator for control and cancellation, Rex for intermediate validation, Vera for end-to-end quality, an AbortSignal chain so no orphaned work runs when nobody is listening, and ENABLE_RAG so the system works with or without a cloud database.